

TP : Segmentation des Communes selon leurs Indicateurs Environnementaux

Objectifs

1. **Appliquer l'Apprentissage Non Supervisé** : Comprendre et mettre en œuvre l'algorithme de **K-Means** pour le *clustering*.
2. **Maîtriser la Standardisation des Données** : Comprendre pourquoi la mise à l'échelle est cruciale pour les algorithmes basés sur la distance.
3. **Déterminer le nombre optimal de clusters** (Méthode du Coude).
4. **Interpréter les segments créés** : Analyser le "profil" de chaque groupe de communes (ex: "Communes émettrices" vs. "Communes vertes").

Outils

Outil	Utilisation
Environnement	Google Colaboratory (Colab)
Librairies Python	Pandas, scikit-learn (StandardScaler, KMeans, silhouette_score), Matplotlib/Seaborn .

Jeu de Données

Les groupes utiliseront la base de données propre de l'Atelier précédent, mais cette fois en se concentrant uniquement sur des **variables indicatives** de l'environnement, sans variable cible Y définie au préalable (puisque l'on découvre les segments).

- **Exemples de Caractéristiques (Features X)** : Émissions de CO2 par habitant, Densité de population, Proportion d'énergies renouvelables, Surface d'espaces verts, Revenu moyen (comme indicateur socio-économique).

Phase 1 : Préparation pour le Clustering

Le *clustering* par K-Means est très sensible à l'échelle des variables.

Étape	Consignes Python Clés	Justification (Pourquoi le fait-on ?)
1. Sélection des Features	Isoler uniquement les colonnes numériques pertinentes pour la segmentation.	Se concentrer sur les indicateurs qui permettront de différencier les communes.
2. Standardisation/Scaling	<pre>from sklearn.preprocessing import StandardScaler ; StandardScaler().fit_transform(X)</pre>	Étape Cruciale : Mettre toutes les variables sur la même échelle (moyenne ≈ 0 , écart-type ≈ 1). Sans cela, une variable avec une grande magnitude (ex: Population) dominerait le calcul de distance, rendant les autres variables inutiles.

Phase 2 : Détermination du Nombre Optimal de Clusters (K)

L'efficacité du K-Means dépend du choix du paramètre K (le nombre de groupes).

1. Méthode du Coude (*Elbow Method*) :

- Exécuter K-Means dans une boucle pour un nombre de clusters K allant de 2 à 10.
- Pour chaque exécution, enregistrer l'**Inertie** (`model.inertia_`), qui représente la somme des carrés des distances entre chaque observation et le centre de son cluster.
- **Visualisation :** Tracer la courbe de l'Inertie en fonction de K. Le point où la descente de l'inertie commence à ralentir de manière significative (le "coude") indique le K optimal.

2. Métriques Alternatives (Approfondissement) :

- Calculer le **Score de Silhouette** (`silhouette_score`) pour chaque K. Un score élevé indique que les clusters sont bien séparés et cohérents en interne.

Phase 3 : Modélisation Finale et Interprétation

1. Entraînement Final :

- Initialiser le modèle K-Means avec la valeur de K optimale trouvée (`model = KMeans(n_clusters=K_optimal, random_state=42)`).
- Entraîner le modèle et assigner les étiquettes de cluster au jeu de données original : `df['Cluster'] = model.labels_`.

2. Interprétation des Profils :

- Calculer le **Centre de chaque Cluster** (`df.groupby('Cluster').mean()`). Cela donne le profil moyen de chaque groupe de communes pour chaque indicateur (ex: Cluster 0 a une forte CO2 et une faible EnR).
- **Visualisation (Heatmap) :** Afficher les moyennes des clusters sous forme de *heatmap* (`sns.heatmap`) pour visualiser rapidement les indicateurs qui sont les plus élevés/faibles dans chaque segment.

3. Visualisation 2D :

- Créer un **Nuage de Points** (`sns.scatterplot`) en utilisant les deux variables les plus importantes (selon l'analyse de la *heatmap*).
- Colorier les points en fonction de leur label de **Cluster** pour visualiser la séparation.